

Buổi 06 (3 tiết) – Chương 3: Component & Hooks (3.1–3.3)

1 Chương 3: Component trong React Native

1.1 3.1 Giới thiệu Component

Component là một trong những khái niệm cốt lõi và quan trọng nhất trong React Native (và React nói chung). Chúng là các khối xây dựng độc lập, có thể tái sử dụng để tạo nên giao diện người dùng (UI). Hãy tưởng tượng ứng dụng của bạn như một ngôi nhà được xây dựng từ nhiều viên gạch khác nhau – mỗi viên gạch chính là một component.

Đặc điểm chính của Component:

- **Tính độc lập (Independent):** Mỗi component hoạt động độc lập và có logic riêng. Bạn có thể phát triển và kiểm thử từng component riêng lẻ.
- **Tính tái sử dụng (Reusable):** Một khi đã tạo, component có thể được sử dụng nhiều lần ở các vị trí khác nhau trong ứng dụng, giúp giảm thiểu việc viết code trùng lặp.
- **Tính đóng gói (Encapsulated):** Component bao gồm cả logic hiển thị (JSX) và hành vi (state, props, hooks) của một phần giao diện cụ thể.
- **Dựa trên Component:** Mọi thứ bạn thấy trên màn hình trong ứng dụng React Native đều là một component, từ nút bấm đơn giản đến một màn hình phức tạp.

Lợi ích khi sử dụng Component:

- **Dễ dàng quản lý và bảo trì code:** Ứng dụng được chia thành các phần nhỏ hơn, dễ hiểu và dễ dàng sửa đổi.
- **Tăng tốc độ phát triển:** Khả năng tái sử dụng component giúp tiết kiệm thời gian và công sức.
- **Cải thiện hiệu suất:** React có cơ chế tối ưu hóa việc render lại các component, giúp ứng dụng chạy mượt mà hơn.
- **Tạo giao diện nhất quán:**

1.2 3.2 Cách tạo Component

Trong React Native, có hai loại component chính:

- **Functional Component (Component hàm):** Đây là loại component đơn giản nhất, là một hàm JavaScript (hoặc arrow function) nhận props (thuộc tính) làm đối số và trả về JSX (JavaScript XML) để mô tả giao diện. Functional component thường được ưu tiên sử dụng cho các thành phần chỉ hiển thị dữ liệu và không có logic phức tạp về trạng thái.

Ví dụ:

```
``javascript import React from 'react'; import { Text, View, StyleSheet }
from 'react-native';
```

```
const Greeting = (props) => { return ( <View style={styles.container}>
<Text style={styles.text}>Xin chào, {props.name}!</Text> </View> );
};
```

```
const styles = StyleSheet.create({ container: {
padding: 10, backgroundColor: '#e0f7fa', borderRadius: 5, }, text: {
fontSize: 16, fontWeight: 'bold', }, });
```

```
export default Greeting; ``
```

- **Class Component (Component lớp):** Đây là component được định nghĩa bằng một lớp JavaScript (ES6) kế thừa từ `React.Component`. Class component có thể quản lý state (trạng thái) và có các phương thức lifecycle (vòng đời) để xử lý các sự kiện khác nhau trong quá trình tồn tại của component. Tuy nhiên, với sự ra đời của Hooks, Functional component đã trở nên mạnh mẽ hơn và thường được ưu tiên hơn.

Ví dụ:

```
``javascript import React, { Component } from 'react'; import {
Text, View, StyleSheet } from 'react-native';
```

```
class Counter extends Component { constructor(props) { super(props);
this.state = { count: 0, }; }
```

```
  increment = () => {
```

```
    this.setState({ count: this.state.count + 1 });
```

```
  };
```

```
  render() {
```

```
    return (
```

```
      <View style={styles.container}>
```

```
        <Text style={styles.text}>Đã đếm: {this.state.count}</Text>
```

```
        <Button title="Tăng" onPress={this.increment} />
```

```
      </View>
```

```
    );
```

```
  }
```

```
}
```

```
const styles = StyleSheet.create({ container: {
padding: 20, alignItems: 'center', }, text: { fontSize: 20, }, });
```

1.3 3.3 Hooks trong React Native

Hooks là các hàm đặc biệt cho phép bạn "móc" vào các tính năng của React (như state và lifecycle) từ bên trong Functional component. Hooks được giới thiệu trong React 16.8 và đã trở thành một cách phổ biến để quản lý state và side effect trong Functional component, giúp code trở nên ngắn gọn và dễ hiểu hơn so với Class component.

Các Hooks phổ biến trong React Native:

- **useState:** Cho phép bạn thêm state vào Functional component. Nó trả về một cặp giá trị: state hiện tại và một hàm để cập nhật state đó.

Ví dụ:

```
``javascript import React, { useState } from 'react'; import {
Text, View, Button, StyleSheet } from 'react-native';
```

```
const CounterHook = () => { const [count, setCount] = useState(0);
```

```
  return (
```

```
    <View style={styles.container}>
```

```
      <Text style={styles.text}>Đã đếm (sử dụng Hook): {count}</Text>
```

```
      <Button title="Tăng" onPress={() => setCount(count + 1)} />
```

```
    </View>
```

```
  );
```

```
};
```

```
const styles = StyleSheet.create({ container: {
padding: 20, alignItems: 'center', }, text: { fontSize: 20, }, });
```

```
export default CounterHook; ``
```

- **useEffect:** Cho phép bạn thực hiện các side effect trong Functional component. Side effect có thể là gọi API, thiết lập subscription, hoặc thay đổi DOM một cách thủ công.

Ví dụ:

```
``javascript import React, { useState, useEffect } from 'react'; import {
Text, View, StyleSheet } from 'react-native';
```

```
const ExampleEffect = () => { const [count, setCount] = useState(0);
```

```
useEffect(() => {
```

```
// Được gọi sau mỗi lần render
```

```
console.log(`Đã render với count: ${count}`);
```

```
// Cleanup function (tùy chọn)
```

```
return () => {
```

```
console.log('Cleanup khi component unmount hoặc trước lần render tiếp theo');
```

```
};
```

```
}, [count]); // Chỉ chạy effect này khi giá trị của 'count' thay đổi
```

```
return (
```

```
<View style={styles.container}>
```

```
<Text style={styles.text}>Giá trị: {count}</Text>
```

```
<Button title="Tăng" onPress={() => setCount(count + 1)} />
```

```
        </View>

    );

};

const styles = StyleSheet.create({
  container: {
    padding: 20, alignItems: 'center', }, text: { fontSize: 20, }, });

export default ExampleEffect; ``
```

- **Các Hooks**

khác: useContext, useRef, useReducer, useCallback, useMemo, useEffect, useDebugValue, và bạn cũng có thể tạo Hooks tùy chỉnh của riêng mình.